MYSQL DATABASE

Complete Classroom Notes & Question Bank

What this book contains

- Concept definitions and step-by-step explanations before queries.
- Original-style question statements followed by MySQL solutions and expected outputs.
- Real-world examples, diagrams, comparison tables, and common mistakes.
- MySQL equivalents for Oracle-only syntax found in the source material.
- Revision coverage from database introduction and data types through joins, database objects, ERmodelling, normalization, and assignment queries.

Contents

1. Introduction to databases and SQL
2. MySQL data types, classifications, and ranges
3. Creating tables and inserting data
4. Retrieving data: selection and projection
5. Case sensitivity
6. Constraints
7. KodNest sample tables and reading outputs
8. Operators: arithmetic, relational, concatenation, aliases
9. Keyword operators: DISTINCT, BETWEEN, IN, NULL, LIKE, AND, OR
10. ORDER BY
11. Single-row functions
12. Date and time functions
13. Aggregate functions
14. GROUP BY
15. HAVING
16. Subqueries
17. Joins
18. SQL command categories and transactions
19. Miscellaneous interview-style queries
20. Database objects
21. Oracle-to-MySQL conversion guide
22. ER diagrams and ER schema
23. Normalization
24. Flat-file model and relational model
25. Assignment questions: STATION, Employee, CITY and COUNTRY
26. Final revision checklist

1. Introduction to databases and SQL

1.1 What is data?

Data is a collection of raw facts and values. Examples include a person's name, age, gender, phone number, email address, product price, and attendance date. By itself, a single value may not provide much meaning, but related data can be organized to answer useful questions.

1.2 What is a database?

A **database** is an organized collection of related data that can be stored, searched, updated, and managed. Examples include a university database, hospital database, bank database, library database, and online shopping database.

1.3 How data was managed before database systems

Before database systems, people recorded information in paper ledgers, registers, filing cabinets, and separate computer files. For example, a college could keep student details in one register, fee payments in another, and exam marks in a third. The same student's name and ID might be copied several times.

Older approach	Problem	Database-based approach
Paper ledger/register	Slow to search, difficult to update, can be damaged or lost	Data is stored in structured tables and queried with SQL
Separate text or program files	Duplicate values, inconsistent copies, manual searching	A DBMS manages shared data and relationships
One program variable per fact	Not practical for a very large number of facts	Rows and columns store large collections of records

1.4 Why DBMS was needed

A **Database Management System (DBMS)** is software that creates, stores, retrieves, updates, protects, and manages databases. It helps control access, reduce inconsistency, enforce rules, and recover data after failures. A relational DBMS stores related data in tables.

```
Paper records / separate files
          |           | problems: duplication, slow search,
inconsistency           v Database Management System (DBMS)
          |
          +--> Tables and relationships
          +--> SQL queries
          +--> Constraints and permissions
          +--> Transactions and recovery
```

1.5 What is SQL?

SQL stands for Structured Query Language. It is used to define relational database structures, insert and modify data, retrieve information, and manage permissions and transactions. SQL is a language; MySQL is a database management system that implements SQL.

1.6 MySQL

MySQL is a relational database management system. It organizes data in tables and supports primary keys, foreign keys, indexes, views, transactions, and many SQL functions.

1.7 History: SQL and MySQL

SQL grew out of relational database research in the 1970s, including work by IBM researchers Donald D. Chamberlin and Raymond F. Boyce. MySQL was created in the 1990s by the Swedish company MySQL AB and became a widely used open-source relational database. The practical lesson is that SQL is the query language, while MySQL is one system that runs SQL statements.

1.8 Structure of a table

Term	Meaning	Example
Table / relation	A structured collection of related rows	student
Row / record / tuple	One complete entry	One student's details
Column / attribute / field	One property shared by rows	student_name
Data type	Defines the kind of value a column accepts	VARCHAR(100), INT
Primary key	Uniquely identifies each row	student_id

```
student +------------+---------------+-----
+
| student_id | student_name  | age |
+------------+---------------+-----+
| 101        | Asha          | 21  |  <-- one row / record
| 102        | Ravi          | 22  |
+------------+---------------+-----+
^              ^     column
column
```

1.9 Basic MySQL commands

```
SHOW DATABASES; CREATE
DATABASE college;
USE college;
SELECT DATABASE();
SHOW TABLES;
```

Expected behavior: SHOW DATABASES lists databases visible to the account; CREATE DATABASE creates college; USE selects it; SELECT DATABASE() returns the selected database; SHOW TABLES lists tables in the selected database.

2. MySQL data types, classifications, and ranges

A data type defines which values a column can store. Choose types according to the meaning of the data, not just the appearance of the value.

2.1 Classification tree

```
MySQL data types
|
+-- Numeric
|   +-- Integers: TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT
|   +-- Exact decimal: DECIMAL, NUMERIC
|   +-- Approximate: FLOAT, DOUBLE
|   +-- Bit: BIT
|
+-- String
|   +-- Character: CHAR, VARCHAR
|   +-- Large text: TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT
|   +-- Binary: BINARY, VARBINARY, BLOB family
|
+-- Date and time
|   +-- DATE, TIME, DATETIME, TIMESTAMP, YEAR
|
+-- Other
    +-- BOOLEAN/BOOL alias, JSON, spatial types
```

2.2 Integer data types and ranges

For an n-bit signed integer, the minimum is −2^(n−1) and the maximum is 2^(n−1)−1. For an unsigned integer, the range is 0 through 2^n−1. Integer types use fixed storage sizes.

Type	Bytes	Signed minimum	Signed maximum	Unsigned maximum
TINYINT	1	−128	127	255
SMALLINT	2	−32,768	32,767	65,535
MEDIUMINT	3	−8,388,608	8,388,607	16,777,215
INT / INTEGER	4	−2,147,483,648	2,147,483,647	4,294,967,295
BIGINT	8	−9,223,372,036,854,775,808	9,223,372,036,854,775,807	18,446,744,073,709,551,615

Example calculation for TINYINT: it uses 8 bits. Signed range is −2^7 through 2^7−1, which is −128 through 127. Unsigned range is 0 through 2^8−1, which is 0 through 255.

Important: INT(10) does not mean an integer that can store only ten digits. The number in parentheses historically related to display width, not the storage range.

2.3 DECIMAL and NUMERIC

DECIMAL(p,s) stores an exact decimal number. **p** is total precision, or total number of digits; **s** is scale, or number of digits after the decimal point. Integer-digit capacity is p − s. For DECIMAL(p,s), the maximum positive magnitude is 10^(p−s) − 10^(−s), and the smallest positive step is 10^(−s).

```
price DECIMAL(8,2)
Precision = 8 total digits
```

```
Scale     = 2 digits after decimal
Digits before decimal = 8 - 2 = 6
Example maximum magnitude: 999999.99
Valid examples: 8.00, 73.69, 61.30
Not representable at scale 2 without rounding: 61.3267
```

Use DECIMAL for money and exact quantities. FLOAT and DOUBLE are approximate and may not store a decimal fraction exactly.

2.4 FLOAT and DOUBLE

FLOAT and DOUBLE store approximate floating-point values. DOUBLE usually has more precision than FLOAT. Their range is not calculated using the signed-integer formula because their binary representation divides bits among sign, exponent, and significand.

2.5 Character and binary types

Type	Definition and use
CHAR(n)	Fixed-length character string. Useful for fixed-width codes. May pad values with spaces.
VARCHAR(n)	Variable-length character string up to n characters. Useful for names, email addresses, and phone numbers.
TINYTEXT / TEXT / MEDIUMTEXT / LONGTEXT	Text types with increasing capacity for longer descriptions.
BINARY(n) / VARBINARY(n)	Fixed- or variable-length bytes, not ordinary human-language text.
TINYBLOB / BLOB / MEDIUMBLOB / LONGBLOB	Binary large objects such as raw file content.

A phone number should normally be VARCHAR, not INT, because it can contain a leading zero, a country prefix, spaces, or a plus sign. It is an identifier, not a value used for arithmetic.

2.6 Date and time types

Type	Meaning	Example
DATE	Calendar date	'2026-10-09'
TIME	Time of day or time-like interval	'14:30:00'
DATETIME	Date and time; not automatically time-zone converted	'2026-10-09 14:30:00'
TIMESTAMP	Date/time stored with session/system time-zone conversion behavior	'2026-10-09 14:30:00'
YEAR	Year value	2026

2.7 BOOLEAN, BIT, JSON, and spatial data

In MySQL, BOOL and BOOLEAN are aliases for TINYINT(1), commonly used with 0 and 1. BIT stores bit values. JSON stores JSON documents. Spatial types such as POINT, LINESTRING, POLYGON, and GEOMETRY support spatial information.

2.8 Create a table using multiple data types

```
CREATE TABLE product (     product_id BIGINT
UNSIGNED PRIMARY KEY,     product_name
VARCHAR(120) NOT NULL,     price DECIMAL(10,2)
NOT NULL,     quantity INT UNSIGNED DEFAULT 0,
active BOOLEAN DEFAULT TRUE,     created_at
DATETIME DEFAULT CURRENT_TIMESTAMP );

DESCRIBE product;
```

DESCRIBE product displays the column names, data types, NULL allowance, key details, default values, and extra attributes.

3. Creating tables and inserting data

The basic workflow is: create a database, select it, create a table, insert rows, and retrieve them. In MySQL, each complete SQL statement ends with a semicolon.

3.1 CREATE TABLE

Definition: CREATE TABLE defines a table's name, columns, data types, and optional constraints.

```
CREATE TABLE student (
id INT,     name
VARCHAR(100),     age
INT,     gender
VARCHAR(20),     branch
VARCHAR(20),     marks
DECIMAL(5,2) );
```

3.2 INSERT

INSERT adds rows to a table. Listing the column names is safer than relying on the table's column order.

```
INSERT INTO student (id, name, age, gender, branch, marks)
VALUES (101, 'Asha', 21, 'Female', 'ECE', 93.00);

INSERT INTO student (id, name, age, gender, branch, marks)
VALUES
(102, 'Ravi', 22, 'Male', 'CSE', 82.50),
(103, 'Meena', 20, 'Female', 'ECE', 88.00);

SELECT * FROM student;
```

id	name	age	gender	branch	marks
101	Asha	21	Female	ECE	93.00
102	Ravi	22	Male	CSE	82.50
103	Meena	20	Female	ECE	88.00

3.3 Inspect a table

```
DESCRIBE student;
SHOW CREATE TABLE student;
SELECT * FROM student LIMIT 10;
```

DESCRIBE shows the structure. SHOW CREATE TABLE displays the full creation statement. LIMIT 10 previews up to ten rows.

4. Retrieving data: selection and projection

Retrieving data means reading rows from a table with SELECT.

4.1 Projection

Projection means selecting particular columns. It changes which columns are displayed, not which rows are included.

QUESTION

Write a query to display name from the student table.

Explanation: Only the name column is required, so place that column after SELECT.

MySQL query:

```
SELECT name FROM student;
```

Expected output / result: One column named name, containing each student's name.

QUESTION

Write a query to display name, age, and gender from the student table.

Explanation: List the three required columns separated by commas.

MySQL query:

```
SELECT name, age, gender FROM student;
```

QUESTION

Write a query to display all data from the student table.

Explanation: The asterisk (*) means all columns.

MySQL query:

```
SELECT * FROM student;
```

4.2 Selection

Selection means filtering rows using a condition in WHERE.

QUESTION

Write a query to display the name of the student whose age is 22.

Explanation: WHERE age = 22 filters rows to students whose age equals 22.

MySQL query:

```
SELECT name FROM student WHERE age = 22;
```

Expected output / result: With the sample rows above, Ravi is returned.

QUESTION

Write a query to display the details of students whose gender is male.

Explanation: Text values are written in quotes. The comparison follows the column's collation rules.

MySQL query:

```
SELECT * FROM student WHERE gender = 'Male';
```

Expected output / result: With the sample rows above, the row for Ravi is returned.

QUESTION

Write a query to display details of students who scored greater than 60 marks.

Explanation: The greater-than operator > excludes marks equal to 60.

MySQL query:

```
SELECT * FROM student WHERE marks > 60;
```

Expected output / result: All three sample rows are returned because each has marks greater than 60.

5. Case sensitivity

Case sensitivity means whether uppercase and lowercase letters are treated as different values. SQL keywords such as SELECT and select are equivalent in normal MySQL usage. Text comparison depends on the collation of the column or expression. Many default MySQL collations are case-insensitive.

```
-- May match Akash, AKASH, or akash under a case-insensitive collation
SELECT * FROM emp WHERE f_name = 'akash';

-- Force case-sensitive byte comparison for this expression
SELECT * FROM emp WHERE BINARY f_name = 'Akash';

-- Inspect column collations
SHOW FULL COLUMNS FROM emp;
```

Important: Do not assume all MySQL installations use the same text comparison rules. Check the actual column collation when exact case behavior matters. Database/table-name case behavior can also vary by operating system and server settings.

6. Constraints

A constraint is a rule enforced by the database to keep stored data valid. Constraints help implement business requirements and prevent missing values, duplicates, invalid values, or broken relationships.

```
Constraint types covered:
  NOT NULL  -> value must be supplied
  UNIQUE    -> value must not duplicate another non-NULL value
  CHECK     -> value must satisfy a condition
  DEFAULT   -> use a fallback value when omitted
  PRIMARY KEY -> uniquely identifies each row
  FOREIGN KEY -> keeps relationships consistent
```

6.1 NOT NULL

Definition: NOT NULL prevents a column from storing SQL NULL. It does not prevent duplicate values.

Real-world example: A registration record must have a name.

```
CREATE TABLE student_required (
id INT,     name VARCHAR(67)
NOT NULL,     age INT,
institute VARCHAR(56) );

INSERT INTO student_required (id, name, age, institute)
VALUES (1, 'Asha', 21, 'KodNest');

SELECT * FROM student_required;
```

id	name	age	institute
1	Asha	21	KodNest

This fails because name is omitted: INSERT INTO student_required (id, age) VALUES (2, 22);

6.2 UNIQUE

Definition: UNIQUE prevents duplicate non-NULL values in a column or set of columns. In MySQL, a UNIQUE index generally permits multiple NULL values.

Real-world example: Account email addresses should not be duplicated.

```
CREATE TABLE account (
account_id INT PRIMARY KEY,
email VARCHAR(150) UNIQUE );

INSERT INTO account VALUES (1, 'asha@example.com');
SELECT * FROM account;
```

Inserting a second row with the same email causes a duplicate-key error.

6.3 PRIMARY KEY

Definition: A primary key uniquely identifies each row. It combines uniqueness and NOT NULL. A table has one primary-key constraint, which can be made of one or multiple columns.

Real-world example: student_id distinguishes two students who share the same name.

```
CREATE TABLE student_pk (
student_id INT PRIMARY KEY,
student_name VARCHAR(100) NOT NULL );

INSERT INTO student_pk VALUES (101, 'Asha');
```

```
SELECT * FROM student_pk;
```

6.4 FOREIGN KEY

Definition: A foreign key requires a value to match a referenced key in a parent table, protecting referential integrity. In MySQL, use a foreign-key-supporting engine such as InnoDB.

Real-world example: An employee should not reference a department that does not exist.

```
CREATE TABLE department (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(80) NOT NULL );

CREATE TABLE employee (     emp_id INT PRIMARY KEY,
emp_name VARCHAR(100) NOT NULL,     dept_id INT,
FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES (20, 'Administration');
INSERT INTO employee VALUES (1, 'Asha', 20);

SELECT e.emp_id, e.emp_name, d.dept_name
FROM employee e
JOIN department d ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
1	Asha	Administration

An employee with dept_id 99 is rejected unless department 99 exists. If the foreign-key column allows NULL, NULL can be used to represent no assigned department.

6.5 CHECK

Definition: CHECK rejects a row when its condition is false. In MySQL, CHECK constraints are enforced starting with version 8.0.16.

Real-world example: A particular membership category might require age greater than 18.

```
CREATE TABLE eligible_person (
id INT PRIMARY KEY,     name
VARCHAR(100) NOT NULL,     age
INT CHECK (age > 18) );

INSERT INTO eligible_person VALUES (1, 'Ravi', 21);
SELECT * FROM eligible_person;
```

The value 21 passes. Age 18 fails because the condition is strictly greater than 18. If the rule is 18 or older, use CHECK (age >= 18).

6.6 DEFAULT

Definition: DEFAULT supplies a value when an INSERT omits a column or uses DEFAULT for that column.

```
CREATE TABLE citizen_default (
id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
gender VARCHAR(20) DEFAULT 'male' );

INSERT INTO citizen_default (id, name) VALUES (1, 'Ravi');
SELECT * FROM citizen_default;
```

id	name	gender
1	Ravi	male

The gender value is 'male' because the INSERT omitted the gender column. The example reproduces the source exercise's specified default; real applications should use a default that matches the actual business requirement.

6.7 Combined constraint question from KodNest

QUESTION

Write a query to create a CITIZEN table with id, name, gender, address, phone_number, and age. Make id the primary key, name NOT NULL, gender DEFAULT 'male', and age CHECK(age > 18).

Explanation: Each requirement is implemented as a column definition or table constraint. A phone number is VARCHAR because it may include a country prefix or leading zero.

MySQL query:

```
CREATE TABLE citizen (     id INT
PRIMARY KEY,     name VARCHAR(100) NOT
NULL,     gender VARCHAR(20) DEFAULT
'male',     address VARCHAR(255),
phone_number VARCHAR(20),     age INT
CHECK (age > 18) );

INSERT INTO citizen (id, name, address, phone_number, age)
VALUES (1, 'Asha', 'Bengaluru', '+91-9876543210', 22);

SELECT * FROM citizen;
```

Expected output / result: One row is returned: 1, Asha, male, Bengaluru, +91-9876543210, 22.

7. KodNest sample tables and reading outputs

The later exercises in the source repeatedly use sample tables. The main names include EMP, DEPT, J_GRADE, STUDENT, COLLEGE_STUDENT, STATION, EMPLOYEE, CITY, and COUNTRY.

Table	Columns referenced by the exercises
EMP	emp_id, f_name, l_name, email, phone_number, hire_date, job_id, salary, commission_pct, manager_id, dept_id
DEPT	dept_id, dept_name, manager_id, loc_id
J_GRADE	grade, low_sal, high_sal
STUDENT	name, age, gender, marks
Table	**Columns referenced by the exercises**
COLLEGE_STUDENT	id, name
STATION	id, city, state, lat_n, long_w
EMPLOYEE (assignment)	employee_id, name, months, salary
CITY	id, name, countrycode, district, population
COUNTRY	code, continent, region, population and other country fields

Before running a query, check the table structure:

```
DESCRIBE emp;
DESCRIBE dept;
SELECT * FROM emp LIMIT 5;
SELECT * FROM dept LIMIT 5;
```

Important: The exact output rows depend on the sample data installed in your MySQL database. The query and its result shape can be explained reliably, but do not copy guessed employee rows as if they were guaranteed outputs.

8. Operators: arithmetic, relational, concatenation, aliases

8.1 Arithmetic operators

Operator	Meaning	Example
+	Addition	salary + 1000
-	Subtraction	salary - 1000
*	Multiplication	salary * 1.10
/	Division	salary / 12
% or MOD()	Remainder	120 % 2 = 0

QUESTION

Write a query to display employee ID, first name, last name, and salary after increasing salary by 1000.

Explanation: The expression salary + 1000 calculates a displayed value; it does not update the stored salary.

MySQL query:

```
SELECT emp_id, f_name, l_name, salary, salary + 1000 AS increased_salary FROM emp;
```

EMP TABLE

Complete source data: 19 employee records. The original EMP_ID gap at 6 is preserved. Phone numbers are stored as text, and NULL indicates a missing value.

EMP_ID	F_NAME	L_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPT_ID
1	Akash	Pandey	akash@gmail.com	9997766	2012-03-29	AD_PRES	34000	NULL	NULL	21
2	Prabhakaran	Ganeshan	pk@gmail.com	77665522	2012-05-25	AD_VP	45000	NULL	100	22
3	DEEP	Nair	deep@gmail.com	876297	2012-04-19	AD_VP	55000	0.50	101	24
4	Ashwin	K	ash@gmail.com	66554433	2011-02-27	IT_PROG	36000	0.75	NULL	110
5	John	Williams	sfda@gmail.com	7456465	2010-12-15	IT_PROG	18000	1.50	110	25
7	Shashi	Raj	Sac@gmail.com	9575673	2010-08-25	ST_MAN	85000	1.20	NULL	22
8	Andy	J	And@gmail.com	8579452	2012-08-28	ST_CLERK	28500	0.90	NULL	22
9	Daniel	King	sas@gmail.com	5423424	2010-06-24	ST_CLERK	12000	1.20	117	23
10	Adam	Sam	adam@gmail.com	325674	2007-04-16	ST_CLERK	53000	1.20	196	110
11	Meghana	Devraj	Meg@gmail.com	3453424	2008-10-25	ST_CLERK	53000	1.20	165	24
12	Madavan	Manish	mad@gmail.com	4353213	2008-12-30	SA_MAN	25000	0.30	198	23
13	Braven	Bhupathi	Brav@gmail.com	12342231	2008-09-27	SA_REP	45500	0.20	114	22
14	Mamatha	T	Mam@gmail.com	53434334	2010-07-25	SA_REP	46000	0.20	NULL	23
15	Savitha	Singh	savi@gmail.com	3443333	2010-02-23	SA_REP	12000	0.15	NULL	24

16	Mrudul	Kumar	mru@gmail.com	322378445	2010-04-30	AD_ASST	19500	1.20	NULL	23
17	Pankaj	patel	pap@gmail.com	322378445	2010-04-23	MD_MAN	39500	1.20	NULL	22
18	Janardhan	Tohn	Tohn@gmail.com	322378445	2011-05-28	MD_REP	29500	1.20	NULL	22
19	Sardhar	L	Sar@gmail.com	322378445	2010-04-29	AC_MGR	29500	1.20	NULL	110
20	Jinnath	Whala	jin@gmail.com	322378445	2010-04-30	AC_ACCOUNT	9500	1.20	NULL	110

Column names match the original KodNest source: emp_id, f_name, l_name, email, phone_number, hire_date, job_id, salary, commission_pct, manager_id, dept_id.

DEPT TABLE

Complete source data: all 9 department records.

DEPT_ID	DEPT_NAME	MANAGER_ID	LOC_ID
20	Admin	300	2100
21	Marketing	301	2200
22	Shipping	302	2300
23	IT	303	2400
24	Sales	304	2500
25	Executive	305	2600
110	Acc	306	2700
190	Contracting	NULL	2800
26	Admin	307	2900

Column meanings: dept_id is the department identifier; dept_name is the department name; manager_id identifies the department manager; loc_id is the location identifier.

J_GRADE TABLE

Complete source data: all 7 salary-grade records exactly as displayed in the source material.

GRADE	LOW_SAL	HIGH_SAL
A	2000	60000
B	9001	19000
D	1001	25000
F	25001	35000
H	35001	89000
I	45001	99000
J	55001	199000

Column meanings: grade is the grade label; low_sal is the lower salary boundary; high_sal is the upper salary boundary. The original source values are retained.

MySQL setup script (1 of 3): create the database and tables

```
CREATE DATABASE IF NOT EXISTS kodnest_sql_practice;
USE kodnest_sql_practice;

CREATE TABLE emp (   emp_id INT
PRIMARY KEY,   f_name VARCHAR(40)
NOT NULL,   l_name VARCHAR(40) NOT
NULL,   email VARCHAR(100),
phone_number VARCHAR(20),
hire_date DATE,   job_id
VARCHAR(30),   salary
DECIMAL(10,2),   commission_pct
DECIMAL(5,2) NULL,   manager_id INT
NULL,   dept_id INT NULL );

CREATE TABLE dept (   dept_id INT
PRIMARY KEY,   dept_name
VARCHAR(60) NOT NULL,
manager_id INT NULL,   loc_id INT
);

CREATE TABLE j_grade (
grade CHAR(1) PRIMARY KEY,
low_sal INT,   high_sal INT
);
```

Column names are intentionally aligned with the source. For example, the correct names are f_name, l_name, hire_date, loc_id, low_sal and high_sal.

MySQL setup script (2 of 3): insert all EMP rows

```
INSERT INTO emp (emp_id, f_name, l_name, email, phone_number,
hire_date, job_id,  salary, commission_pct, manager_id, dept_id)
VALUES
(1, 'Akash', 'Pandey', 'akash@gmail.com',
 '9997766', '2012-03-29', 'AD_PRES', 34000, NULL, NULL, 21),
(2, 'Prabhakaran', 'Ganeshan', 'pk@gmail.com',
 '77665522', '2012-05-25', 'AD_VP', 45000, NULL, 100, 22),
(3, 'DEEP', 'Nair', 'deep@gmail.com',
 '876297', '2012-04-19', 'AD_VP', 55000, 0.50, 101, 24),
(4, 'Ashwin', 'K', 'ash@gmail.com',
 '66554433', '2011-02-27', 'IT_PROG', 36000, 0.75, NULL, 110),
(5, 'John', 'Williams', 'sfda@gmail.com',
 '7456465', '2010-12-15', 'IT_PROG', 18000, 1.50, 110, 25),
(7, 'Shashi', 'Raj', 'Sac@gmail.com',
 '9575673', '2010-08-25', 'ST_MAN', 85000, 1.20, NULL, 22),
(8, 'Andy', 'J', 'And@gmail.com',
 '8579452', '2012-08-28', 'ST_CLERK', 28500, 0.90, NULL, 22),
(9, 'Daniel', 'King', 'sas@gmail.com',
 '5423424', '2010-06-24', 'ST_CLERK', 12000, 1.20, 117, 23),
(10, 'Adam', 'Sam', 'adam@gmail.com',
 '325674', '2007-04-16', 'ST_CLERK', 53000, 1.20, 196, 110),
(11, 'Meghana', 'Devraj', 'Meg@gmail.com',
 '3453424', '2008-10-25', 'ST_CLERK', 53000, 1.20, 165, 24),
(12, 'Madavan', 'Manish', 'mad@gmail.com',
 '4353213', '2008-12-30', 'SA_MAN', 25000, 0.30, 198, 23),
(13, 'Braven', 'Bhupathi', 'Brav@gmail.com',
 '12342231', '2008-09-27', 'SA_REP', 45500, 0.20, 114, 22),
(14, 'Mamatha', 'T', 'Mam@gmail.com',
 '53434334', '2010-07-25', 'SA_REP', 46000, 0.20, NULL, 23),
(15, 'Savitha', 'Singh', 'savi@gmail.com',
 '3443333', '2010-02-23', 'SA_REP', 12000, 0.15, NULL, 24),
(16, 'Mrudul', 'Kumar', 'mru@gmail.com',
 '322378445', '2010-04-30', 'AD_ASST', 19500, 1.20, NULL, 23),
(17, 'Pankaj', 'patel', 'pap@gmail.com',
 '322378445', '2010-04-23', 'MD_MAN', 39500, 1.20, NULL, 22),
(18, 'Janardhan', 'Tohn', 'Tohn@gmail.com',
 '322378445', '2011-05-28', 'MD_REP', 29500, 1.20, NULL, 22),
(19, 'Sardhar', 'L', 'Sar@gmail.com',
 '322378445', '2010-04-29', 'AC_MGR', 29500, 1.20, NULL, 110),
(20, 'Jinnath', 'Whala', 'jin@gmail.com',
 '322378445', '2010-04-30', 'AC_ACCOUNT', 9500, 1.20, NULL, 110);
```

MySQL setup script (3 of 3): insert DEPT and J_GRADE rows

```
INSERT INTO dept (dept_id, dept_name, manager_id, loc_id) VALUES
(20, 'Admin', 300, 2100),
(21, 'Marketing', 301, 2200),
(22, 'Shipping', 302, 2300),
(23, 'IT', 303, 2400),
(24, 'Sales', 304, 2500),
(25, 'Executive', 305, 2600),
(110, 'Acc', 306, 2700),
(190, 'Contracting', NULL, 2800),
(26, 'Admin', 307, 2900);

INSERT INTO j_grade (grade, low_sal, high_sal) VALUES
('A', 2000, 60000),
('B', 9001, 19000),
('D', 1001, 25000),
('F', 25001, 35000),
('H', 35001, 89000),
('I', 45001, 99000),
('J', 55001, 199000);

-- Check that all source rows were loaded
SELECT * FROM emp ORDER BY emp_id;
SELECT * FROM dept ORDER BY dept_id;
SELECT * FROM j_grade ORDER BY grade;
```

Expected row counts after setup: EMP = 19 rows, DEPT = 9 rows, J_GRADE = 7 rows. EMP_ID 6 is intentionally absent because it is absent from the source table.

Before practicing: use `DESCRIBE emp;`, `DESCRIBE dept;`, and `DESCRIBE j_grade;` to verify the schema.

QUESTION

Write a query to display employee ID, email, phone number, and salary after reducing salary by 1000.

Explanation: Subtract 1000 from salary and give the expression a readable alias.

MySQL query:

```
SELECT emp_id, email, phone_number, salary - 1000 AS reduced_salary FROM emp;
```

QUESTION

Write a query to display employee ID, job ID, department ID, and salary after a 10% hike.

Explanation: A 10% increase means original salary plus 10% of salary, or salary multiplied by 1.10.

MySQL query:

```
SELECT emp_id, job_id, dept_id, salary, salary * 1.10 AS hiked_salary FROM emp;
```

QUESTION

Write a query to display employee ID, job ID, department ID, and salary after a 20% reduction.

Explanation: A 20% reduction leaves 80% of the original salary.

MySQL query:

```
SELECT emp_id, job_id, dept_id, salary, salary * 0.80 AS reduced_salary FROM emp;
```

QUESTION

Write a query to display half-yearly salary.

Explanation: Assuming salary is monthly salary, multiply it by six.

MySQL query:

```
SELECT emp_id, salary, salary * 6 AS half_yearly_salary FROM emp;
```

8.2 Operator precedence

Parentheses are evaluated first. Multiplication, division, and modulo are evaluated before addition and subtraction. Use parentheses when the intended order might be unclear.

```
SELECT 10 + 2 * 3 AS result;    -- 16
SELECT (10 + 2) * 3 AS result;  -- 36
SELECT 20 / 5 * 2 AS result;    -- 8
```

8.3 Relational operators

Operator	Meaning
=	Equal
<> or !=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal

<=	Less than or equal

QUESTION

Write a query to display employees whose first name is Akash.

Explanation: WHERE filters employee rows by exact name, subject to the column collation.

MySQL query:

```
SELECT * FROM emp WHERE f_name = 'Akash';
```

QUESTION

Write a query to display employees whose department ID is greater than 30.

Explanation: The > operator excludes 30 itself.

MySQL query:

```
SELECT * FROM emp WHERE dept_id > 30;
```

QUESTION

Write a query to display employees whose salary is less than 10000.

Explanation: The condition is applied to each row before results are returned.

MySQL query:

```
SELECT * FROM emp WHERE salary < 10000;
```

QUESTION

Write a query to display email and phone number for employees whose commission percentage is greater than or equal to 5.

Explanation: Use >= for an inclusive boundary. Confirm whether commission_pct stores whole percentages or fractions in the given dataset.

MySQL query:

```
SELECT email, phone_number FROM emp WHERE commission_pct >= 5;
```

Write a query to display employee ID, first name, and last name where salary is less than or equal to 34000.

Explanation: The <= operator includes employees earning exactly 34000.

MySQL query:

```
SELECT emp_id, f_name, l_name FROM emp WHERE salary <= 34000;
```

QUESTION

Write a query to display employees who are not clerks (job_id is not ST_CLERK).

Explanation: Use <> for not equal. Rows where job_id is NULL do not satisfy this condition.

MySQL query:

```
SELECT * FROM emp WHERE job_id <> 'ST_CLERK';
```

QUESTION

8.4 CONCAT and aliases

MySQL uses CONCAT() to join strings. The result of an expression can be labelled using AS. Unlike Oracle, MySQL does not require FROM DUAL for simple literal expressions.

QUESTION

Write a query to concatenate first name and last name from EMP and display the result as full_name.

Explanation: CONCAT joins the values. Add a space as a separate argument so the names do not run together.

MySQL query:

```
SELECT CONCAT(f_name, ' ', l_name) AS full_name FROM emp;
```

QUESTION

Write a query to concatenate 'Sachin' and 'Tendulkar' as full name.

Explanation: Use a literal space between the two names.

MySQL query:

```
SELECT CONCAT('Sachin', ' ', 'Tendulkar') AS full_name;
```

Expected output / result: One value: Sachin Tendulkar.

QUESTION

Write a query to concatenate '111' and '222' as full number.

Explanation: The values are strings, so CONCAT joins them without arithmetic.

MySQL query:

```
SELECT CONCAT('111', '222') AS full_number;
```

Expected output / result: 111222.

QUESTION

Write a query to concatenate 'Bond' and '7777' as movie character.

Explanation: CONCAT can combine text and digit characters.

MySQL query:

```
SELECT CONCAT('Bond', '7777') AS movie_character;
```

Expected output / result: Bond7777.

QUESTION

Write a query to concatenate 'Bond' with NULL and display the result.

Explanation: In MySQL, CONCAT returns NULL if any argument is NULL.

MySQL query:

```
SELECT CONCAT('Bond', NULL) AS result;
```

Expected output / result: NULL.

8.5 DISTINCT

DISTINCT removes duplicate rows from the selected columns.

QUESTION

Write a query to display unique department IDs from EMP.

Explanation: DISTINCT returns each department ID only once.

MySQL query:

```
SELECT DISTINCT dept_id FROM emp;
```

QUESTION

Write a query to display distinct salaries from EMP.

Explanation: Repeated salary values are collapsed to one result value.

MySQL query:

```
SELECT DISTINCT salary FROM emp;
```

8.6 BETWEEN and NOT BETWEEN

BETWEEN includes both boundary values. NOT BETWEEN selects values outside the inclusive range.

QUESTION

Display grade and low salary where low salary is between 2000 and 4000.

Explanation: Both 2000 and 4000 are included.

QUESTION

MySQL query:

```
SELECT grade, low_sal FROM j_grade WHERE low_sal BETWEEN 2000 AND 4000;
```

Display employees whose commission percentage is between 1.2 and 1.5.

Explanation: BETWEEN includes 1.2 and 1.5. Check the scale used by commission_pct in the sample data.

MySQL query:

```
SELECT * FROM emp WHERE commission_pct BETWEEN 1.2 AND 1.5;
```

QUESTION

Display department ID and department name where location ID is not between 2400 and 2500.

Explanation: NOT BETWEEN excludes values from 2400 through 2500, including both boundaries.

MySQL query:

```
SELECT dept_id, dept_name FROM dept WHERE loc_id NOT BETWEEN 2400 AND 2500;
```

QUESTION

Display job grades whose high salary is not between 3000 and 55000.

Explanation: Rows with high_sal below 3000 or above 55000 qualify.

MySQL query:

```
SELECT * FROM j_grade WHERE high_sal NOT BETWEEN 3000 AND 55000;
```

8.7 IN and NOT IN

IN checks membership in a list. NOT IN excludes values in the list. If a NOT IN list or subquery contains NULL, SQL's three-valued logic can produce surprising results.

QUESTION

Display employees whose salary is 12000, 18000, or 19000.

Explanation: IN is shorter than multiple OR comparisons.

MySQL query:

```
SELECT * FROM emp WHERE salary IN (12000, 18000, 19000);
```

QUESTION

Display department ID and department name where location ID is 2500, 2700, or 2900.

Explanation: The row qualifies when loc_id matches any listed value.

MySQL query:

QUESTION

```
SELECT dept_id, dept_name FROM dept WHERE loc_id IN (2500, 2700, 2900);
```

Display low and high salaries for grades other than A, B, and D.

Explanation: NOT IN excludes those three grade labels.

MySQL query:

```
SELECT low_sal, high_sal FROM j_grade WHERE grade NOT IN ('A', 'B', 'D');
```

8.8 NULL

NULL represents missing or unknown information. It is not the same as zero or an empty string. Use IS NULL and IS NOT NULL.

QUESTION

Display employees who do not take any commission.

Explanation: A missing commission is represented by NULL, so test it using IS NULL.

MySQL query:

```
SELECT * FROM emp WHERE commission_pct IS NULL;
```

QUESTION

Display employees who have a manager.

Explanation: Employees with a manager have a non-NULL manager_id.

MySQL query:

```
SELECT * FROM emp WHERE manager_id IS NOT NULL;
```

8.9 LIKE and wildcards

Pattern	Meaning
A%	Starts with A
%ng	Ends with ng
%an%	Contains an
_r%	Second character is r
__@%	Third character is @
_____	Exactly five characters

QUESTION

Display employees whose first name starts with A.

Explanation: Percent (%) matches zero or more characters.

QUESTION

MySQL query:

```
SELECT * FROM emp WHERE f_name LIKE 'A%';
```

Display departments whose name starts with AC.

Explanation: AC must be at the beginning of dept_name.

MySQL query:

```
SELECT dept_id, dept_name FROM dept WHERE dept_name LIKE 'AC%';
```

QUESTION

Display grades whose low salary ends with 1.

Explanation: CAST converts the numeric salary to text so LIKE can test the last character.

MySQL query:

```
SELECT grade FROM j_grade WHERE CAST(low_sal AS CHAR) LIKE '%1';
```

QUESTION

Display departments whose name ends with 'ng'.

Explanation: The pattern begins with % so any prefix is allowed.

MySQL query:

```
SELECT * FROM dept WHERE dept_name LIKE '%ng';
```

QUESTION

Display employees whose first name contains 'an'.

Explanation: The pattern permits any text before and after an.

MySQL query:

```
SELECT * FROM emp WHERE f_name LIKE '%an%';
```

QUESTION

Display last names containing 'am'.

Explanation: LIKE is a pattern comparison; case sensitivity depends on collation.

MySQL query:

```
SELECT l_name FROM emp WHERE l_name LIKE '%am%';
```

QUESTION

Display employees whose first name has 'r' as the second character.

QUESTION

Explanation: Underscore matches exactly one character.

MySQL query:

```
SELECT * FROM emp WHERE f_name LIKE '_r%';
```

Display employees whose email has @ as the third character.

Explanation: Two underscores match the first two characters, so @ is tested at position three.

MySQL query:

```
SELECT * FROM emp WHERE email LIKE '__@%';
```

QUESTION

Display employees whose last name has exactly five characters.

Explanation: Five underscore characters require exactly five characters in the pattern.

MySQL query:

```
SELECT l_name, salary FROM emp WHERE l_name LIKE '_____';
```

8.10 Searching for literal underscore and percent

Inside LIKE, underscore and percent are wildcards. To search for a literal underscore or percent, escape the wildcard.

```
SELECT name FROM college_student
WHERE name LIKE '%\\_%';

SELECT name FROM college_student
WHERE name LIKE '%\\%%';
```

The first query searches for names containing a literal underscore. The second searches for names containing a literal percent sign. Backslash is the default LIKE escape character in MySQL.

8.11 AND and OR

AND requires all combined conditions to be true. OR requires at least one condition to be true. AND has higher precedence than OR, so parentheses make combined conditions easier to reason about.

QUESTION

Display employees whose salary is 12000 and job ID is ST_CLERK.

Explanation: Both conditions must be true.

MySQL query:

```
SELECT * FROM emp WHERE salary = 12000 AND job_id = 'ST_CLERK';
```

QUESTION
QUESTION

Display employees whose department ID is 22 and manager ID is 100.

Explanation: Both department and manager conditions are required.

MySQL query:

```
SELECT * FROM emp WHERE dept_id = 22 AND manager_id = 100;
```

Display grades where grade is A or high salary is greater than 80000.

Explanation: Either condition can qualify the row.

MySQL query:

```
SELECT * FROM j_grade WHERE grade = 'A' OR high_sal > 80000;
```

QUESTION

Display employees who are ST_CLERK or earn less than 19000.

Explanation: An employee qualifies if either the job ID condition or salary condition is true.

MySQL query:

```
SELECT * FROM emp WHERE job_id = 'ST_CLERK' OR salary < 19000;
```

QUESTION

Display employees who are president or sales representatives and earn more than 25000.

Explanation: Parentheses group the two job IDs; salary must also exceed 25000.

MySQL query:

```
SELECT * FROM emp WHERE job_id IN ('AD_PRES', 'SA_REP') AND salary > 25000;
```

QUESTION

9. ORDER BY

ORDER BY sorts the result. ASC is ascending and is the default; DESC is descending. Without ORDER BY, row order is not guaranteed.

QUESTION

Display all employee first names in alphabetical order.

Explanation: Sort the result by f_name ascending.

MySQL query:

```
SELECT f_name FROM emp ORDER BY f_name ASC;
```

QUESTION

Display hire dates from oldest to newest.

Explanation: Earlier dates come first in ascending date order.

MySQL query:

```
SELECT hire_date FROM emp ORDER BY hire_date ASC;
```

Display salaries in descending order.

Explanation: The highest salary appears first.

MySQL query:

```
SELECT salary FROM emp ORDER BY salary DESC;
```

QUESTION

Display commission_pct in ascending order.

Explanation: ASC sorts low to high; NULL placement follows MySQL's sort behavior.

MySQL query:

```
SELECT commission_pct FROM emp ORDER BY commission_pct ASC;
```

QUESTION

Display commission_pct in descending order.

Explanation: DESC sorts high to low.

MySQL query:

```
SELECT commission_pct FROM emp ORDER BY commission_pct DESC;
```

QUESTION

10. Single-row functions

A single-row function produces a result for each row or input expression. Common groups include string functions, numeric functions, date/time functions, NULL-handling functions, and conditional functions.

10.1 String functions

Function	Purpose	Example
UPPER(s)	Convert text to uppercase	UPPER('Akash') -> AKASH
LOWER(s)	Convert text to lowercase	LOWER('Pandey') -> pandey
CONCAT(...)	Join strings	CONCAT('A','B') -> AB
CHAR_LENGTH(s)	Count characters	CHAR_LENGTH('Ravi') -> 4
SUBSTRING(s,pos,len)	Extract characters; position begins at 1	SUBSTRING('PRABHAKARAN',2,5) -> RABHA
LOCATE(a,b)	Find a string's position, or 0 if absent	LOCATE('a','Pandey') -> 2
TRIM()	Remove characters/spaces at edges	TRIM(' hi ') -> hi
LPAD/RPAD	Pad left/right to a requested length	LPAD('Sharma',10,'#') -> ####Sharma
REPLACE(s,old,new)	Replace occurrences	REPLACE('apple','p','&') -> a&≤

Display uppercase of the text 'Akash'.

Explanation: UPPER converts letters to uppercase. MySQL does not need FROM DUAL for this expression.

MySQL query:

```
SELECT UPPER('Akash') AS result;
```

Expected output / result: AKASH.

QUESTION

Display lowercase of the text 'Pandey'.

Explanation: LOWER converts letters to lowercase.

MySQL query:

QUESTION

```
SELECT LOWER('Pandey') AS result;
```

Expected output / result: pandey.

QUESTION

Display the initial letter capitalized in 'kodnest'.

Explanation: Capitalize the first character with UPPER(LEFT(...)) and lowercase the rest.

MySQL query:

```
SELECT CONCAT(UPPER(LEFT('kodnest', 1)), LOWER(SUBSTRING('kodnest', 2))) AS result;
```

Expected output / result: Kodnest.

QUESTION

Display all employee first names in uppercase.

Explanation: Apply UPPER to the column.

MySQL query:

```
SELECT UPPER(f_name) AS f_name FROM emp;
```

QUESTION

Display first and last names in uppercase.

Explanation: Apply UPPER to each column independently.

MySQL query:

```
SELECT UPPER(f_name) AS f_name, UPPER(l_name) AS l_name FROM emp;
```

Display names and salary where first name is Akash in any case.

Explanation: LOWER normalizes the value before comparing it. A case-insensitive collation may also make the simple comparison case-insensitive.

MySQL query:

```
SELECT f_name, l_name, salary FROM emp WHERE LOWER(f_name) = 'akash';
```

QUESTION

Concatenate 'Akash' and 'Pandey'.

Explanation: CONCAT joins the two strings with a space between them.

MySQL query:

```
SELECT CONCAT('Akash', ' ', 'Pandey') AS full_name;
```

Expected output / result: Akash Pandey.

QUESTION
QUESTION

Concatenate first and last name for all employees.

Explanation: The space is included as a separate argument.

MySQL query:

```
SELECT CONCAT(f_name, ' ', l_name) AS full_name FROM emp;
```

QUESTION

Combine the strings '1111' and '2222'.

Explanation: String concatenation joins them; it does not add their numeric values.

MySQL query:

```
SELECT CONCAT('1111', '2222') AS result;
```

Expected output / result: 11112222.

QUESTION

Combine the string '1111' and 'Ramu'.

Explanation: CONCAT accepts strings with different content.

MySQL query:

```
SELECT CONCAT('1111', 'Ramu') AS result;
```

Expected output / result: 1111Ramu.

Combine '1111' with NULL.

Explanation: CONCAT returns NULL if any argument is NULL.

MySQL query:

```
SELECT CONCAT('1111', NULL) AS result;
```

Expected output / result: NULL.

QUESTION

Find the length of the string 'Prabhakaran'.

Explanation: CHAR_LENGTH counts characters rather than bytes.

MySQL query:

```
SELECT CHAR_LENGTH('Prabhakaran') AS length;
```

Expected output / result: 11.

QUESTION
QUESTION

Find the length of every employee first name.

Explanation: Return the original value and its character count.

MySQL query:

```
SELECT f_name, CHAR_LENGTH(f_name) AS name_length FROM emp;
```

QUESTION

Find the length of '1111'.

Explanation: Each digit character counts as one character.

MySQL query:

```
SELECT CHAR_LENGTH('1111') AS length;
```

Expected output / result: 4.

QUESTION

Find the length of NULL.

Explanation: A function given NULL returns NULL here.

MySQL query:

```
SELECT CHAR_LENGTH(NULL) AS length;
```

Expected output / result: NULL.

Display five characters from 'PRABHAKARAN' starting at position 2.

Explanation: MySQL substring positions begin at 1.

MySQL query:

```
SELECT SUBSTRING('PRABHAKARAN', 2, 5) AS result;
```

Expected output / result: RABHA.

QUESTION

Extract 'oha' from 'RajaRamMohanRoy'.

Explanation: Use SUBSTRING with the starting position and length. The substring begins at position 9.

MySQL query:

```
SELECT SUBSTRING('RajaRamMohanRoy', 9, 3) AS result;
```

Expected output / result: oha.

QUESTION
QUESTION

Find the position of 'a' in 'Pandey'.

Explanation: LOCATE returns a 1-based position, or 0 if not found.

MySQL query:

```
SELECT LOCATE('a', 'Pandey') AS position;
```

Expected output / result: 2.

QUESTION

Find the position of 'a' in every employee first name.

Explanation: Apply LOCATE to each row.

MySQL query:

```
SELECT f_name, LOCATE('a', f_name) AS position_of_a FROM emp;
```

QUESTION

Extract 'bha' from the first name Prabhakaran.

Explanation: Find where 'bha' begins, then extract three characters.

MySQL query:

```
SELECT SUBSTRING(f_name, LOCATE('bha', f_name), 3) AS result FROM emp WHERE f_name = 'Prabhakaran';
```

Expected output / result: bha, assuming that row exists with that exact first name.

Trim the leading 'm' characters from 'malayalam'.

Explanation: TRIM(LEADING ... FROM ...) removes matching characters at the beginning.

MySQL query:

```
SELECT TRIM(LEADING 'm' FROM 'malayalam') AS result;
```

Expected output / result: alayalam.

QUESTION

Trim the trailing 'm' characters from 'malayalam'.

Explanation: TRIM(TRAILING ...) removes matching characters at the end.

MySQL query:

```
SELECT TRIM(TRAILING 'm' FROM 'malayalam') AS result;
```

Expected output / result: malayala.

QUESTION
QUESTION

Trim 'm' characters from both ends of 'malayalam'.

Explanation: TRIM(BOTH ...) removes matching characters from the start and end.

MySQL query:

```
SELECT TRIM(BOTH 'm' FROM 'malayalam') AS result;
```

Expected output / result: alayala.

QUESTION

Display 'Sharma' as '####Sharma'.

Explanation: LPAD adds padding on the left until the requested length is reached.

MySQL query:

```
SELECT LPAD('Sharma', 10, '#') AS result;
```

Expected output / result: ####Sharma.

QUESTION

Display 'Sharma' as 'Sharma####'.

Explanation: RPAD adds padding on the right.

MySQL query:

```
SELECT RPAD('Sharma', 10, '#') AS result;
```

Expected output / result: Sharma####.

Display 'Sharma' as 'Sharma#!#!'.

Explanation: RPAD repeats the pad string as needed and truncates to the requested length.

MySQL query:

```
SELECT RPAD('Sharma', 10, '#!') AS result;
```

Expected output / result: Sharma#!#!.

QUESTION

Replace every lowercase p in employee first names with an ampersand.

Explanation: REPLACE substitutes all occurrences of the specified substring.

MySQL query:

```
SELECT REPLACE(f_name, 'p', '&') AS changed_name FROM emp;
```

QUESTION

10.2 Numeric functions

Function	Definition	Example
ROUND(n,d)	Round to d decimal places; negative d rounds left of decimal point	ROUND(45.326,2) = 45.33
TRUNCATE(n,d)	Discard digits beyond d places without rounding	TRUNCATE(45.326,2) = 45.32
MOD(a,b)	Return remainder	MOD(120,2) = 0
FORMAT(n,d)	Return formatted text with separators and d decimals	FORMAT(1234.5,2) = 1,234.50

QUESTION

Round 45.326 to two decimal places.

Explanation: ROUND rounds the next digit to determine the final retained digit.

MySQL query:

```
SELECT ROUND(45.326, 2) AS result;
```

Expected output / result: 45.33.

QUESTION

Round 1234.356 to one decimal place.

Explanation: The hundredths digit is 5, so the tenths digit rounds up.

MySQL query:

```
SELECT ROUND(1234.356, 1) AS result;
```

Expected output / result: 1234.4.

Round 2678.345 to the nearest hundred.

Explanation: A negative second argument rounds to the left of the decimal point.

MySQL query:

```
SELECT ROUND(2678.345, -2) AS result;
```

Expected output / result: 2700.

QUESTION

Truncate 45.326 to two decimal places.

Explanation: TRUNCATE discards extra digits instead of rounding.

QUESTION

MySQL query:

```
SELECT TRUNCATE(45.326, 2) AS result;
```

Expected output / result: 45.32.

QUESTION

Find the remainder after dividing 120 by 2.

Explanation: MOD returns the remainder after division.

MySQL query:

```
SELECT MOD(120, 2) AS result;
```

Expected output / result: 0.

10.3 NULL and conditional functions

QUESTION

Display commission_pct, replacing NULL with 0.

Explanation: COALESCE returns the first non-NULL argument.

MySQL query:

```
SELECT COALESCE(commission_pct, 0) AS commission_pct FROM emp;
```

QUESTION

Display 0 when commission_pct is NULL, otherwise display 5.

Explanation: IF(condition, true_value, false_value) is a MySQL conditional function.

MySQL query:

```
SELECT IF(commission_pct IS NULL, 0, 5) AS commission_value FROM emp;
```

QUESTION

Display first name, last name, and name lengths. If both lengths are equal, show NULL; otherwise show the first-name length.

Explanation: Compare CHAR_LENGTH values in IF. The condition is evaluated per row.

MySQL query:

```
SELECT f_name, l_name, CHAR_LENGTH(f_name) AS first_length, CHAR_LENGTH(l_name) AS last_length, IF(
```

QUESTION

11. Date and time functions

MySQL date/time functions operate on DATE, TIME, DATETIME, and TIMESTAMP values. Use ISO-style date literals such as '2014-02-14' to avoid ambiguous formats.

Function	Purpose	Example
CURRENT_DATE()	Current date	SELECT CURRENT_DATE();
TIMESTAMPDIFF(unit,start,end)	Difference in requested units	TIMESTAMPDIFF(MONTH,'2014-02-14','2014-11-14') -> 9
DATE_ADD()	Add an interval	DATE_ADD('2014-02-14', INTERVAL 9 MONTH)
DATE_SUB()	Subtract an interval	DATE_SUB('2014-02-14', INTERVAL 9 MONTH)
DAYOFWEEK()	Sunday=1 through Saturday=7	DAYOFWEEK('2012-12-08') -> 7
LAST_DAY()	Last day of a month	LAST_DAY('2022-06-18') -> 2022-06-30
DATE_FORMAT()	Format date as text	DATE_FORMAT(hire_date,'%d/%m')

QUESTION

Display the system date.

Explanation: CURRENT_DATE() returns the current date according to the MySQL server/session environment.

MySQL query:

```
SELECT CURRENT_DATE() AS system_date;
```

Expected output / result: A date such as 2026-10-09; the exact result depends on the day you run it.

QUESTION

Find the number of whole months between 14-Feb-2014 and 14-Nov-2014.

Explanation: Use TIMESTAMPDIFF with MONTH and place the earlier date first, later date second.

MySQL query:

```
SELECT TIMESTAMPDIFF(MONTH, '2014-02-14', '2014-11-14') AS months_between;
```

Expected output / result: 9.

Add nine months to 14-Feb-2014.

Explanation: DATE_ADD adds the specified interval.

QUESTION

MySQL query:

```
SELECT DATE_ADD('2014-02-14', INTERVAL 9 MONTH) AS result;
```

Expected output / result: 2014-11-14.

QUESTION

Subtract nine months from 14-Feb-2014.

Explanation: DATE_SUB subtracts the specified interval.

MySQL query:

```
SELECT DATE_SUB('2014-02-14', INTERVAL 9 MONTH) AS result;
```

Expected output / result: 2013-05-14.

QUESTION

Find the next Saturday after 08-Dec-2012.

Explanation: The source date is already a Saturday, so this query returns the following Saturday, seven days later.

MySQL query:

```
SELECT DATE_ADD('2012-12-08', INTERVAL 7 DAY) AS next_saturday;
```

Expected output / result: 2012-12-15.

QUESTION

Find the last day of the month containing 18-Jun-2022.

Explanation: LAST_DAY returns the final calendar date of that month.

MySQL query:

```
SELECT LAST_DAY('2022-06-18') AS month_end;
```

Expected output / result: 2022-06-30.

QUESTION

Display last name and hire date as day/month.

Explanation: DATE_FORMAT uses %d for day and %m for month.

MySQL query:

```
SELECT l_name, DATE_FORMAT(hire_date, '%d/%m') AS hire_day_month FROM emp;
```

Display last name and salary with a dollar sign and two decimal places.

Explanation: FORMAT creates a formatted string; it is for display, not numeric calculations.

MySQL query:

```
SELECT l_name, CONCAT('$', FORMAT(salary, 2)) AS formatted_salary FROM emp;
```

QUESTION

12. Aggregate functions

Aggregate functions summarize multiple rows. They often return one result for the whole table or one result per GROUP BY group.

Function	Meaning	NULL behavior
COUNT(*)	Counts rows	Counts rows even when some columns are NULL
COUNT(column)	Counts non-NULL values	Ignores NULL in that column
COUNT(DISTINCT column)	Counts distinct non-NULL values	Duplicates removed
MIN(column)	Smallest value	Ignores NULL
MAX(column)	Largest value	Ignores NULL
SUM(column)	Total	Ignores NULL; returns NULL if no non-NULL values
AVG(column)	Average	Ignores NULL; returns NULL if no non-NULL values

QUESTION

Display the count of all rows in EMP.

Explanation: COUNT(*) counts every row.

MySQL query:

```
SELECT COUNT(*) AS employee_count FROM emp;
```

QUESTION

Display the count of distinct salaries in EMP.

Explanation: DISTINCT removes duplicate salary values before counting.

MySQL query:

```
SELECT COUNT(DISTINCT salary) AS distinct_salary_count FROM emp;
```

Display the minimum salary in EMP.

Explanation: MIN returns the smallest non-NULL salary.

MySQL query:

```
SELECT MIN(salary) AS minimum_salary FROM emp;
```

QUESTION
QUESTION

Display the minimum salary excluding duplicates.

Explanation: DISTINCT does not change a minimum, because duplicate values cannot change the smallest value.

MySQL query:

```
SELECT MIN(DISTINCT salary) AS minimum_salary FROM emp;
```

Expected output / result: Same minimum as MIN(salary).

QUESTION

Display the earliest hire date in EMP.

Explanation: MIN works on date values as well as numbers.

MySQL query:

```
SELECT MIN(hire_date) AS earliest_hire_date FROM emp;
```

QUESTION

Display the maximum salary in EMP.

Explanation: MAX returns the greatest non-NULL salary.

MySQL query:

```
SELECT MAX(salary) AS maximum_salary FROM emp;
```

QUESTION

Display the maximum salary excluding duplicates.

Explanation: DISTINCT does not change the maximum.

MySQL query:

```
SELECT MAX(DISTINCT salary) AS maximum_salary FROM emp;
```

Expected output / result: Same maximum as MAX(salary).

Display the latest hire date in EMP.

Explanation: MAX returns the latest date value.

MySQL query:

```
SELECT MAX(hire_date) AS latest_hire_date FROM emp;
```

QUESTION

Display the sum of all salaries in EMP.

Explanation: SUM adds non-NULL salary values.

QUESTION

MySQL query:

```
SELECT SUM(salary) AS total_salary FROM emp;
```

QUESTION

Display the sum of salaries excluding duplicate salary values.

Explanation: SUM(DISTINCT salary) adds each distinct salary amount once.

MySQL query:

```
SELECT SUM(DISTINCT salary) AS sum_distinct_salary FROM emp;
```

QUESTION

Display the average salary in EMP.

Explanation: AVG calculates the arithmetic mean of non-NULL salary values.

MySQL query:

```
SELECT AVG(salary) AS average_salary FROM emp;
```

QUESTION

Display the average of distinct salaries.

Explanation: AVG(DISTINCT salary) averages each distinct salary value once.

MySQL query:

```
SELECT AVG(DISTINCT salary) AS average_distinct_salary FROM emp;
```

QUESTION

Display the hire date of the oldest employee.

Explanation: The oldest employee by hire date is the employee with the earliest hire_date.

MySQL query:

```
SELECT hire_date FROM emp ORDER BY hire_date ASC LIMIT 1;
```

13. GROUP BY

GROUP BY creates groups of rows that share the same value. Aggregate functions calculate one value per group. A real-world example is calculating total salary by department.

QUESTION

Display department ID and total salary for each department.

Explanation: GROUP BY dept_id forms one group per department, then SUM totals salary in each group.

MySQL query:

```
SELECT dept_id, SUM(salary) AS total_salary FROM emp GROUP BY dept_id;
```

QUESTION

Display department ID and the lowest salary in each department.

Explanation: MIN is calculated separately for each department group.

MySQL query:

```
SELECT dept_id, MIN(salary) AS lowest_salary FROM emp GROUP BY dept_id;
```

QUESTION

Display department ID and highest salary for departments whose ID is greater than 50.

Explanation: WHERE filters rows before grouping. MAX then returns the highest salary for each remaining department.

MySQL query:

```
SELECT dept_id, MAX(salary) AS highest_salary FROM emp WHERE dept_id > 50 GROUP BY dept_id;
```

QUESTION

Display department ID and count of employees in department 90.

Explanation: Filter to department 90 first, then count the remaining rows.

MySQL query:

```
SELECT dept_id, COUNT(*) AS employee_count FROM emp WHERE dept_id = 90 GROUP BY dept_id;
```

Important: With ONLY_FULL_GROUP_BY enabled, each selected non-aggregate expression must normally appear in GROUP BY or be functionally dependent on grouped columns.

14. HAVING

WHERE filters individual rows before groups are formed. HAVING filters groups after GROUP BY. Conditions involving aggregate results usually belong in HAVING.

QUESTION

Display department ID and maximum salary for departments whose maximum salary is greater than 30000.

Explanation: HAVING evaluates the maximum after grouping.

MySQL query:

```
SELECT dept_id, MAX(salary) AS maximum_salary FROM emp GROUP BY dept_id HAVING MAX(salary) > 30000;
```

QUESTION

Display department ID and maximum salary where dept_id is greater than 10 and maximum salary is greater than 30000; sort department IDs descending.

Explanation: WHERE filters employee rows first. GROUP BY groups them. HAVING filters groups. ORDER BY sorts the final groups.

MySQL query:

```
SELECT dept_id, MAX(salary) AS maximum_salary FROM emp WHERE dept_id > 10 GROUP BY dept_id HAVING M
```

QUESTION

Display department ID and minimum salary for ST_CLERK or IT_PROG employees where each group's total salary is below 25000; sort descending by department ID.

Explanation: Filter to the two job IDs, group by department, use HAVING on SUM, then sort.

MySQL query:

```
SELECT dept_id, MIN(salary) AS minimum_salary FROM emp WHERE job_id IN ('ST_CLERK', 'IT_PROG') GROU
```

15. Subqueries

A subquery is a SELECT statement nested inside another SQL statement. Its result can be used as a single value, a list of values, or a test for whether matching rows exist.

```
Outer query
   |    +-- runs / uses result of inner query
(subquery)
          |           +-- SELECT ... FROM
... WHERE ...
```

QUESTION

Display last name, first name, and salary for employees earning more than Pandey.

Explanation: The inner query obtains Pandey's salary. This scalar version assumes only one matching Pandey row.

MySQL query:

```
SELECT l_name, f_name, salary FROM emp WHERE salary > (SELECT salary FROM emp WHERE l_name = 'Pande
```

Note: If several employees share the last name Pandey, choose a clear rule such as comparing with MAX(salary), or use a multi-row operator.

QUESTION

Display department ID and first name for employees working in the same department as Prabhakaran.

Explanation: The subquery returns department IDs for rows matching the first name; IN supports multiple values.

MySQL query:

```
SELECT dept_id, f_name FROM emp WHERE dept_id IN (SELECT dept_id FROM emp WHERE f_name = 'Prabhakar
```

QUESTION

Display department ID, first name, and job ID for employees who work in Administration.

Explanation: The subquery finds the department ID for the department name; the outer query finds employees in that department.

MySQL query:

```
SELECT dept_id, f_name, job_id FROM emp WHERE dept_id IN (SELECT dept_id FROM dept WHERE dept_name
```

QUESTION

Display employee IDs whose department IDs exist in the DEPT table.

Explanation: IN checks whether an employee's department ID is present in the department table.

MySQL query:

```
SELECT emp_id FROM emp WHERE dept_id IN (SELECT dept_id FROM dept);
```

QUESTION

Display last name and job ID for employees earning more than King.

Explanation: The scalar subquery retrieves King's salary. Confirm which name column contains King in your sample schema.

MySQL query:

```
SELECT l_name, job_id FROM emp WHERE salary > (SELECT salary FROM emp WHERE l_name = 'King');
```

Note: If King is stored in f_name instead, use WHERE f_name = 'King'. If multiple rows match, use an appropriate aggregate or multi-row comparison.

16. Joins

A JOIN combines rows from related tables. The example below links EMP to DEPT using dept_id.



Join type	Meaning
INNER JOIN	Returns only matching rows from both tables

Join type	Meaning
LEFT JOIN	Returns all rows from the left table and matching right rows; unmatched right values are NULL
RIGHT JOIN	Returns all rows from the right table and matching left rows
FULL OUTER JOIN	Returns matched and unmatched rows from both sides; MySQL has no direct FULL OUTER JOIN syntax
CROSS JOIN	Returns every possible pair of rows
NATURAL JOIN	Automatically joins using every identically named column; use carefully

QUESTION

Display all details from table1 and table2 where table1.column2 equals table2.column2 using an inner join.

Explanation: INNER JOIN keeps only pairs satisfying the ON condition.

MySQL query:

```
SELECT t1.*, t2.* FROM table1 t1 INNER JOIN table2 t2 ON t1.column2 = t2.column2;
```

QUESTION

Display all details from table1 and table2 where table1.column2 equals table2.column2 using a left outer join.

Explanation: Every table1 row appears; unmatched table2 columns are NULL.

MySQL query:

```
SELECT t1.*, t2.* FROM table1 t1 LEFT JOIN table2 t2 ON t1.column2 = t2.column2;
```

QUESTION

Display all details from table1 and table2 using a right outer join.

Explanation: Every table2 row appears; unmatched table1 columns are NULL.

MySQL query:

```
SELECT t1.*, t2.* FROM table1 t1 RIGHT JOIN table2 t2 ON t1.column2 = t2.column2;
```

QUESTION

Display all details from table1 and table2 using a cross join.

Explanation: Every row in table1 is paired with every row in table2. If table1 has 3 rows and table2 has 4 rows, the result has 12 rows.

MySQL query:

```
SELECT t1.*, t2.* FROM table1 t1 CROSS JOIN table2 t2;
```

QUESTION

Display a full outer join result in MySQL.

Explanation: MySQL does not support FULL OUTER JOIN directly. Combine a left join with unmatched right-side rows.

MySQL query:

```
SELECT t1.*, t2.*
FROM table1 t1
LEFT JOIN table2 t2 ON t1.column2 = t2.column2
UNION ALL
SELECT t1.*, t2.*
FROM table1 t1
RIGHT JOIN table2 t2 ON t1.column2 = t2.column2
WHERE t1.id IS NULL;
```

Note: Replace t1.id with a guaranteed non-NULL primary-key column from table1. UNION ALL retains all rows from both branches.

QUESTION

Display employee ID, first name, salary, and department name for employees whose department IDs match.

Explanation: Join EMP to DEPT on dept_id and select the required columns.

MySQL query:

```
SELECT e.emp_id, e.f_name, e.salary, d.dept_name FROM emp e INNER JOIN dept d ON e.dept_id = d.dept
```

QUESTION

Give an example of a natural join.

Explanation: NATURAL JOIN uses all columns with the same name in both tables. Explicit ON is often safer in real applications.

MySQL query:

```
SELECT * FROM emp NATURAL JOIN dept;
```

QUESTION

Perform a cross join between EMP and DEPT.

Explanation: Every employee row is paired with every department row.

MySQL query:

```
SELECT e.*, d.* FROM emp e CROSS JOIN dept d;
```

QUESTION

Perform a left outer join between EMP and DEPT.

Explanation: Keep every EMP row even if it has no matching department.

MySQL query:

```
SELECT e.*, d.* FROM emp e LEFT JOIN dept d ON e.dept_id = d.dept_id;
```

QUESTION

Perform a right outer join between EMP and DEPT.

Explanation: Keep every DEPT row even if no employee is assigned to it.

MySQL query:

```
SELECT e.*, d.* FROM emp e RIGHT JOIN dept d ON e.dept_id = d.dept_id;
```

QUESTION

Perform a full outer join between EMP and DEPT in MySQL.

Explanation: Use LEFT JOIN plus unmatched DEPT rows. The right-side unmatched branch identifies rows without an EMP match.

MySQL query:

```
SELECT e.*, d.*
FROM emp e
LEFT JOIN dept d ON e.dept_id = d.dept_id
UNION ALL
SELECT e.*, d.*
FROM emp e
RIGHT JOIN dept d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

17. SQL command categories and transactions

Category	Purpose	Statements
DDL	Define or change structures	CREATE, ALTER, DROP, TRUNCATE
DML	Insert, update, delete data	INSERT, UPDATE, DELETE
DQL	Retrieve data	SELECT
DCL	Manage privileges	GRANT, REVOKE
TCL	Manage transactions	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

17.1 DDL examples

```
CREATE TABLE demo (id INT PRIMARY KEY, name VARCHAR(50));
ALTER TABLE demo ADD COLUMN age INT;
TRUNCATE TABLE demo;
DROP TABLE demo;
```

CREATE makes an object. ALTER changes its structure. TRUNCATE removes all rows but retains the table. DROP removes the object.

17.2 DML examples

```
INSERT INTO demo (id, name, age) VALUES (1, 'Ravi', 21);
UPDATE demo SET age = 22 WHERE id = 1;
DELETE FROM demo WHERE id = 1;
```

Important: Check WHERE conditions before UPDATE or DELETE. Omitting WHERE may affect every row.

17.3 DCL and TCL

```
GRANT SELECT ON training.* TO 'reader'@'localhost';
REVOKE SELECT ON training.* FROM 'reader'@'localhost';

START TRANSACTION;
UPDATE demo SET age = 23 WHERE id = 1;
COMMIT;

START TRANSACTION;
UPDATE demo SET age = 24 WHERE id = 1;
ROLLBACK;
```

COMMIT makes uncommitted transaction changes permanent. ROLLBACK cancels uncommitted changes. Transaction support depends on the storage engine; InnoDB supports transactions.

18. Miscellaneous interview-style queries

QUESTION

Get first names from EMP after replacing lowercase 'p' with '&'.

Explanation: REPLACE substitutes every occurrence of p in each first name.

MySQL query:

```
SELECT REPLACE(f_name, 'p', '&') AS changed_name FROM emp;
```

QUESTION

Calculate 35% of salary for Pandey, 10% for Ganeshan, and 15% for everyone else.

Explanation: CASE selects the percentage based on last name. The result is calculated, not stored.

MySQL query:

```
SELECT f_name, l_name, salary,
salary * CASE          WHEN l_name =
'Pandey' THEN 0.35
         WHEN l_name = 'Ganeshan' THEN 0.10
         ELSE 0.15
       END AS calculated_amount
FROM emp;
```

QUESTION

Display the two highest distinct salaries.

Explanation: Sort distinct salary values descending and limit the result to two rows.

MySQL query:

```
SELECT DISTINCT salary FROM emp ORDER BY salary DESC LIMIT 2;
```

QUESTION

Display the second-highest distinct salary.

Explanation: OFFSET 1 skips the highest distinct salary and LIMIT 1 returns the next value.

MySQL query:

```
SELECT DISTINCT salary FROM emp ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

QUESTION

Given employee1 and employee2, return all values from both tables but include common records only once.

Explanation: UNION removes duplicate result rows. Both SELECT statements must return compatible columns.

MySQL query:

```
SELECT id, name FROM employee1 UNION SELECT id, name FROM employee2;
```

QUESTION

Fetch only common records from employee1 and employee2.

Explanation: An INNER JOIN returns records with matching IDs. EXISTS avoids duplicates caused by repeated matches in employee2.

MySQL query:

```
SELECT e1.* FROM employee1 e1 WHERE EXISTS (SELECT 1 FROM employee2 e2 WHERE e2.id = e1.id);
```

QUESTION

Retrieve records from employee1 that are not present in employee2.

Explanation: NOT EXISTS finds rows without a matching key in the second table.

MySQL query:

```
SELECT e1.* FROM employee1 e1 WHERE NOT EXISTS (SELECT 1 FROM employee2 e2 WHERE e2.id = e1.id);
```

19. Database objects

19.1 Views

A view is a named SELECT statement that behaves like a virtual table. It can simplify repeated queries and limit the columns or rows exposed to a user.

```
CREATE VIEW high_salary_employees AS
SELECT emp_id, f_name, salary
FROM emp
WHERE salary > 30000;

SELECT * FROM high_salary_employees;
DROP VIEW high_salary_employees;
```

19.2 Indexes

An index can speed up lookups, joins, and sorting, but uses storage and adds overhead to inserts, updates, and deletes.

```
CREATE INDEX idx_emp_dept_id ON emp(dept_id);
SHOW INDEX FROM emp;
DROP INDEX idx_emp_dept_id ON emp;
```

19.3 Stored procedures

A stored procedure is a named group of SQL statements stored in the database and invoked with CALL.

```
DELIMITER //
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM emp;
END //
DELIMITER ;

CALL GetEmployees();
DROP PROCEDURE GetEmployees;
```

19.4 Triggers

A trigger runs automatically for a table event such as BEFORE INSERT or AFTER UPDATE. Triggers are often used for validation or audit logging.

```
DELIMITER //
CREATE TRIGGER before_emp_insert
BEFORE INSERT ON emp
FOR EACH ROW
BEGIN
    SET NEW.f_name = TRIM(NEW.f_name);
END //
DELIMITER ;

DROP TRIGGER before_emp_insert;
```

19.5 Sequence and AUTO_INCREMENT

The source describes Oracle sequences. For ordinary generated integer IDs, MySQL commonly uses AUTO_INCREMENT.

```
CREATE TABLE generated_id (     id BIGINT UNSIGNED
AUTO_INCREMENT PRIMARY KEY,     name VARCHAR(100)
NOT NULL );
```

```
INSERT INTO generated_id (name) VALUES ('Asha');
SELECT * FROM generated_id;
```

19.6 Synonyms

Oracle supports CREATE SYNONYM. MySQL does not provide an exact direct equivalent with the same semantics. Depending on the goal, a view or a fully qualified table name may be suitable, but they are not identical replacements.

20. Oracle-to-MySQL conversion guide

The KodNest syllabus compares Oracle and MySQL. The following table gives MySQL syntax for concepts that often appear in the exercises.

Oracle-style syntax	MySQL approach
VARCHAR2(n)	VARCHAR(n)
NUMBER	Choose INT, BIGINT, DECIMAL, FLOAT, or DOUBLE according to the values
SELECT expression FROM DUAL	SELECT expression;
a || b (string concatenation)	CONCAT(a, b)
ADD_MONTHS(d, n)	DATE_ADD(d, INTERVAL n MONTH)
MONTHS_BETWEEN(a,b)	TIMESTAMPDIFF(MONTH, b, a), after checking desired argument order and semantics
TO_CHAR(date, format)	DATE_FORMAT(date, format)
TRUNC(number, places)	TRUNCATE(number, places)
Oracle sequence for IDs	AUTO_INCREMENT for common table-ID generation
FULL OUTER JOIN	LEFT JOIN UNION ALL unmatched rows from the right side
ROWNUM / top-N patterns	ORDER BY ... LIMIT ... OFFSET ...

Important: Syntax conversions can differ in edge cases. Always test date calculations, NULL handling, collation, and numeric precision against the intended result.

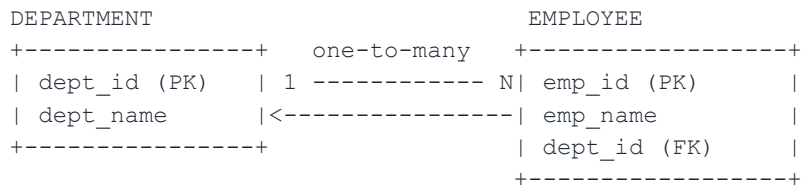
21. ER diagrams and ER schema

An Entity-Relationship diagram visually models entities, their attributes, and their relationships before tables are created.

21.1 Main elements

Element	Definition	Example
Entity	Thing about which information is stored	Student, Course, Department
Attribute	Property of an entity	student_name, course_title
Primary key	Attribute(s) that uniquely identify a record	student_id
Relationship	Association between entities	Student enrolls in Course
Cardinality	Number of entity instances that can participate	1:1, 1:M, M:N

21.2 Relationship diagram



A department can have many employees. The foreign key is stored on the many side, in EMPLOYEE.

21.3 ER schema in MySQL

```
CREATE TABLE department (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(100) NOT NULL );

CREATE TABLE employee (     emp_id INT PRIMARY KEY,
emp_name VARCHAR(100) NOT NULL,     dept_id INT,
FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

An ER schema translates the conceptual diagram into relational tables, columns, data types, primary keys, foreign keys, and constraints.

22. Normalization

Normalization organizes tables to reduce unnecessary duplication and prevent insertion, update, and deletion anomalies. It depends on keys and functional dependencies, not simply on making many tables.

22.1 Why normalization?

Anomaly	Definition
Insertion anomaly	A fact cannot be stored without adding an unrelated fact.
Update anomaly	The same fact is repeated and every copy must be updated consistently.
Deletion anomaly	Removing one record unintentionally removes another fact.

22.2 First Normal Form (1NF)

Definition: Each cell contains one value for the intended attribute, and repeating groups are not stored as lists inside a single cell.

```
Not recommended: student_id | student_name |
courses 1           | Asha          | Java,
SQL, Python

Better: one student-course relationship per row
student_id | course 1           | Java
1          | SQL
1          | Python
```

22.3 Second Normal Form (2NF)

Definition: A table is in 1NF and every non-key attribute depends on the whole candidate key, not only part of a composite key.

If (student_id, course_id) is the key but student_name depends only on student_id, student_name belongs in STUDENT rather than ENROLLMENT.

```
CREATE TABLE student (     student_id
INT PRIMARY KEY,     student_name
VARCHAR(100) NOT NULL );

CREATE TABLE enrollment (     student_id
INT,     course_id INT,     PRIMARY KEY
(student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id)
);
```

22.4 Third Normal Form (3NF)

Definition: A table is in 2NF and non-key attributes do not depend transitively on a candidate key. Informally, non-key values should depend on the key, the whole key, and nothing but the key.

If employee_id determines dept_id and dept_id determines dept_name, store dept_name in DEPARTMENT rather than repeating it for every employee.

22.5 Boyce-Codd Normal Form (BCNF)

Definition: For every non-trivial functional dependency X → Y, X must be a superkey. BCNF is stricter than 3NF and handles some dependency patterns that 3NF can allow.

22.6 Practical normalized example

```
CREATE TABLE department (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(100) NOT NULL );

CREATE TABLE employee (     emp_id INT PRIMARY KEY,
emp_name VARCHAR(100) NOT NULL,     dept_id INT,
FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

23. Flat-file model and relational model

23.1 Flat-file model

A flat-file model stores information in a single simple file, often without explicit relationships or integrity rules. CSV files are common examples.

student_id	student_name	course	instructor_phone
1	Asha	Java	555-0101
1	Asha	SQL	555-0101
student_id	**student_name**	**course**	**instructor_phone**
2	Ravi	Java	555-0101

Repeating student and instructor details can cause duplication and inconsistent updates.

23.2 Relational model

The relational model organizes data into relations, commonly displayed as tables. Rows are tuples, columns are attributes, and keys identify rows. Foreign keys connect related tables.

Term	Definition
Relation	Table-like set of rows with named attributes
Tuple	Row
Attribute	Column
Domain	Allowed values for an attribute
Candidate key	Minimal set of attributes that uniquely identifies a row
Primary key	Chosen candidate key for identifying rows
Foreign key	Attribute(s) referencing a key in another or the same table
Integrity constraint	Rule that valid data must satisfy

24. Assignment questions: STATION, Employee, CITY and COUNTRY

The source's final assignment pages use the STATION, EMPLOYEE, CITY, and COUNTRY schemas. The following questions are reproduced in readable form and solved using MySQL syntax.

24.1 STATION table

Column	Meaning
ID	Station identifier
CITY	City name
STATE	State name
LAT_N	Latitude
LONG_W	Longitude

QUESTION

Query a list of CITY and STATE from the STATION table.

Explanation: Select the two requested columns.

MySQL query:

```
SELECT CITY, STATE FROM STATION;
```

QUESTION

Let N be the number of CITY entries and N' be the number of distinct city names. Find N - N'.

Explanation: COUNT(CITY) ignores NULL city values. If the intended N is every row, use COUNT(*) instead.

MySQL query:

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY) AS difference FROM STATION;
```

QUESTION

Query city names that start with a vowel (A, E, I, O, U).

Explanation: REGEXP tests a regular expression; ^ anchors the pattern to the start of the city name.

MySQL query:

```
SELECT CITY FROM STATION WHERE CITY REGEXP '^[AEIOUaeiou]';
```

QUESTION

Query city names that have a vowel as the second character.

Explanation: The dot matches the first character and the bracket expression tests the second.

MySQL query:

```
SELECT CITY FROM STATION WHERE CITY REGEXP '^.[AEIOUaeiou]';
```

QUESTION

Query city names that do not start with a vowel.

Explanation: NOT REGEXP excludes city names matching the vowel-at-start pattern.

MySQL query:

```
SELECT CITY FROM STATION WHERE CITY NOT REGEXP '^[AEIOUaeiou]';
```

24.2 EMPLOYEE assignment table

The assignment uses employee_id, name, months, and salary.

QUESTION

Print employee names for employees earning more than 2000 per month who have worked fewer than 10 months. Sort by ascending employee ID.

Explanation: Both conditions must be true. ORDER BY employee_id ASC sorts from the lowest ID to the highest.

MySQL query:

```
SELECT name FROM Employee WHERE salary > 2000 AND months < 10 ORDER BY employee_id ASC;
```

24.3 CITY and COUNTRY

CITY includes id, name, countrycode, district, and population. COUNTRY includes code, continent, region, and population.

QUESTION

Find the sum of populations of all cities in countries located in Asia.

Explanation: Join CITY.countrycode to COUNTRY.code, filter countries by continent, and sum city population.

MySQL query:

```
SELECT SUM(ci.population) AS total_population FROM CITY ci JOIN COUNTRY co ON ci.countrycode = co.c
```

24.4 Review the complete assignment set

• SELECT and filtering exercises are covered in Chapter 4.

• Constraint creation exercise is covered in Chapter 6.

• All operator questions are covered in Chapter 8.

• String, numeric, and date functions are covered in Chapters 10 and 11.

• Aggregate functions, GROUP BY, and HAVING are covered in Chapters 12 through 14.

• Subqueries and join questions are covered in Chapters 15 and 16.

• Database objects, ER diagrams, and normalization are covered in Chapters 19 through 22.

QUESTION

25. Final revision checklist

- I can explain what data, database, DBMS, SQL, and MySQL mean.
- I can create a database and table, insert rows, inspect structure, and select records.
- I know integer ranges and how to calculate signed and unsigned limits.
- I can explain NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, and FOREIGN KEY withexamples.
- I can use WHERE, arithmetic and relational operators, DISTINCT, BETWEEN, IN, LIKE, NULL checks,AND, and OR.
- I can explain why MySQL uses CONCAT() rather than assuming Oracle's || concatenation behavior.
- I can use ORDER BY, string functions, numeric functions, date functions, and aggregate functions.
- I understand the difference between WHERE and HAVING and can use GROUP BY.
- I can write scalar and multi-row subqueries.
- I can explain INNER, LEFT, RIGHT, CROSS, NATURAL, and MySQL's workaround for FULL OUTERJOIN.
- I can explain DDL, DML, DQL, DCL, TCL, transactions, views, indexes, procedures, and triggers.
- I can draw a simple ER diagram and explain 1NF, 2NF, 3NF, and BCNF.

Important: Use the sample rows actually installed in your MySQL environment when checking outputs. Some original employee outputs depend on the KodNest sample data; the SQL is given, but this guide does not invent database rows that were not supplied as reliable text.

26. Additional question-by-question practice from the original pages

This chapter captures further exercises visible in the KodNest source pages. Each question is stated first, followed by its MySQL query and expected result or interpretation.

QUESTION

Write a query to display the manager IDs for departments whose department ID begins with digit 2.

Explanation: dept_id is numeric, so cast it to text before applying a LIKE pattern. This is a pattern exercise, not a numeric range test.

MySQL query:

```
SELECT manager_id FROM dept WHERE CAST(dept_id AS CHAR) LIKE '2%';
```

QUESTION

Display department ID and department name for departments whose manager ID ends with 0.

Explanation: Cast manager_id to text so LIKE can check its final character.

MySQL query:

```
SELECT dept_id, dept_name FROM dept WHERE CAST(manager_id AS CHAR) LIKE '%0';
```

QUESTION

Display employees whose last name has the fourth character from the end equal to n.

Explanation: A four-character suffix pattern requires any prefix followed by three characters, then n at the fourth-from-last position. REGEXP makes the intent clearer.

MySQL query:

```
SELECT * FROM emp WHERE l_name REGEXP 'n...$';
```

Expected output / result: Rows whose last name ends with n followed by exactly three characters match; this means n is fourth from the end.

QUESTION

Display phone numbers whose third digit from the end is 3.

Explanation: The last two characters can be any digits, but the third from the end must be 3.

MySQL query:

```
SELECT phone_number FROM emp WHERE phone_number REGEXP '3[0-9]{2}$';
```

Expected output / result: If phone_number contains punctuation or spaces, normalize it first or use a suitable string expression.

QUESTION

Create COLLEGE_STUDENT and insert sample names containing underscore and percent characters.

Explanation: Use VARCHAR for IDs and names. The escape character lets LIKE search for a literal wildcard.

MySQL query:

```
CREATE TABLE college_student (
id VARCHAR(10) PRIMARY KEY,
name VARCHAR(100) );

INSERT INTO college_student (id, name) VALUES
('Kod001', 'Sahana_varma'),
('Kod002', 'Sindhu_sharma'),
('Kod003', 'Swathi%hegde'),
('Kod004', 'Shruthi');

SELECT name FROM college_student WHERE name LIKE '%\\_%';
SELECT name FROM college_student WHERE name LIKE '%\\%%';
```

Expected output / result: The first query returns names containing a literal underscore; the second returns names containing a literal percent sign.

QUESTION

Display the grade where grade is A or high salary is greater than 80000.

Explanation: OR means either condition can be true.

MySQL query:

```
SELECT * FROM j_grade WHERE grade = 'A' OR high_sal > 80000;
```

QUESTION

Display employees who are president, sales representative, or earn more than 35000.

Explanation: This exercise uses OR across all three alternatives.

MySQL query:

```
SELECT * FROM emp WHERE job_id = 'AD_PRES' OR job_id = 'SA_REP' OR salary > 35000;
```

QUESTION

Display the output of ROUND(45.326, 2).

Explanation: ROUND retains two digits after the decimal point and rounds using the next digit.

MySQL query:

```
SELECT ROUND(45.326, 2) AS result;
```

Expected output / result: 45.33.

QUESTION

Display the output of ROUND(1234.356, 1).

Explanation: One digit is retained after the decimal point; the next digit causes rounding.

MySQL query:

```
SELECT ROUND(1234.356, 1) AS result;
```

Expected output / result: 1234.4.

QUESTION

Display the output of ROUND(2678.345, -2).

Explanation: A negative precision rounds to the nearest hundred.

MySQL query:

```
SELECT ROUND(2678.345, -2) AS result;
```

Expected output / result: 2700.

QUESTION

Display the output of ROUND(123, -2).

Explanation: Round 123 to the nearest hundred.

MySQL query:

```
SELECT ROUND(123, -2) AS result;
```

Expected output / result: 100.

QUESTION

Convert Oracle TRUNC(45.326, 2) to MySQL and show the output.

Explanation: MySQL's TRUNCATE function removes digits after the specified scale without rounding.

MySQL query:

```
SELECT TRUNCATE(45.326, 2) AS result;
```

Expected output / result: 45.32.

QUESTION

Convert Oracle TRUNC(1234.356, 1) to MySQL and show the output.

Explanation: Retain one digit after the decimal point and discard the rest.

MySQL query:

QUESTION

```
SELECT TRUNCATE(1234.356, 1) AS result;
```

Expected output / result: 1234.3.

Convert Oracle TRUNC(2678.345, -2) to MySQL and show the output.

Explanation: Negative decimal places truncate to the left of the decimal point.

MySQL query:

```
SELECT TRUNCATE(2678.345, -2) AS result;
```

Expected output / result: 2600.

QUESTION

Convert Oracle TRUNC(123, -2) to MySQL and show the output.

Explanation: Truncate the tens and ones digits.

MySQL query:

```
SELECT TRUNCATE(123, -2) AS result;
```

Expected output / result: 100.

QUESTION

Display the output of MOD(120, 2).

Explanation: MOD returns the remainder after division.

MySQL query:

```
SELECT MOD(120, 2) AS result;
```

Expected output / result: 0.

QUESTION

Display the department ID and department name where location ID is 2500, 2700, or 2900.

Explanation: Use IN for a list of permitted location IDs.

MySQL query:

```
SELECT dept_id, dept_name FROM dept WHERE loc_id IN (2500, 2700, 2900);
```

QUESTION

Display details of employees whose salary is 12000 and job ID is ST_CLERK.

Explanation: Both conditions must match.

QUESTION

MySQL query:

```
SELECT * FROM emp WHERE salary = 12000 AND job_id = 'ST_CLERK';
```

Display details of employees whose department ID is 22 and manager ID is 100.

Explanation: Use AND because both conditions are required.

MySQL query:

```
SELECT * FROM emp WHERE dept_id = 22 AND manager_id = 100;
```

QUESTION

Display the details of employees whose first name contains 'an'.

Explanation: The % wildcard allows any characters before and after the substring.

MySQL query:

```
SELECT * FROM emp WHERE f_name LIKE '%an%';
```

QUESTION

Display the last name and salary for employees whose last name contains exactly five characters.

Explanation: Five underscore wildcards require five characters.

MySQL query:

```
SELECT l_name, salary FROM emp WHERE l_name LIKE '_____';
```

QUESTION

Convert Oracle's string concatenation and DUAL examples to MySQL.

Explanation: MySQL can select literal expressions without FROM DUAL and uses CONCAT for string concatenation.

MySQL query:

```
SELECT CONCAT('Akash', ' ', 'Pandey') AS full_name;
SELECT CONCAT('1111', '2222') AS combined_value;
SELECT ROUND(45.326, 2) AS rounded_value;
```

Expected output / result: Results are Akash Pandey, 11112222, and 45.33.